DEEP LEARNING

# RNNs & LSTMs Visual Guide

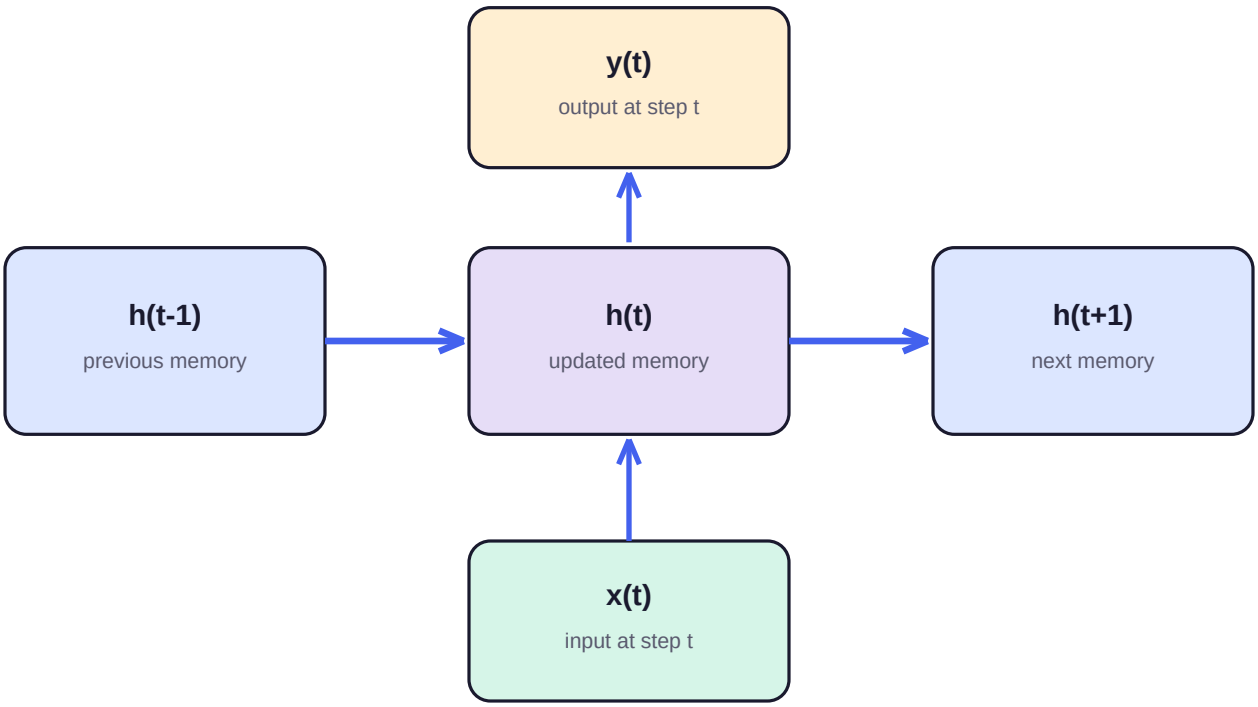
Networks with memory:
sequence processing, vanishing gradients, LSTM gates.

**ANIMATED WALKTHROUGH**
Step through pages to play the animation

STEP 1/5

# Sequences, One Step at a Time

The hidden state carries memory forward.

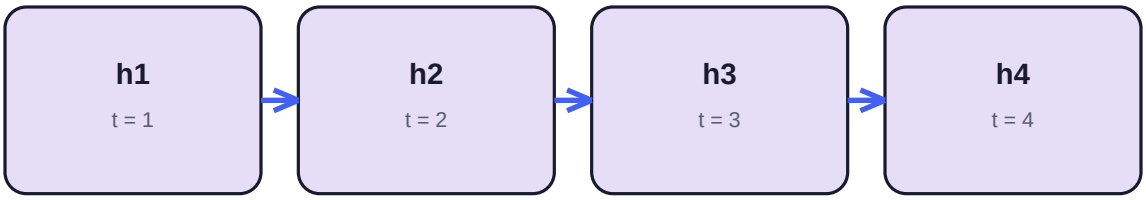



Same weights reused at every timestep. The network's memory IS the hidden state h.

STEP 2/5

# Unrolled Through Time

The loop becomes a very deep feedforward net.



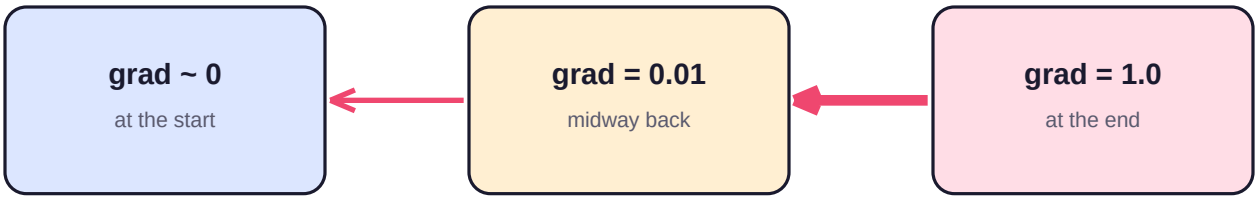

Unrolling exposes the loop as a chain. Backprop through it is called backprop-through-time (BPTT).

A 100-step sequence unrolls into a 100-layer network — which is exactly why gradients misbehave.

STEP 3/5

# The Vanishing Gradient Problem

Gradients shrink (or explode) across many steps.



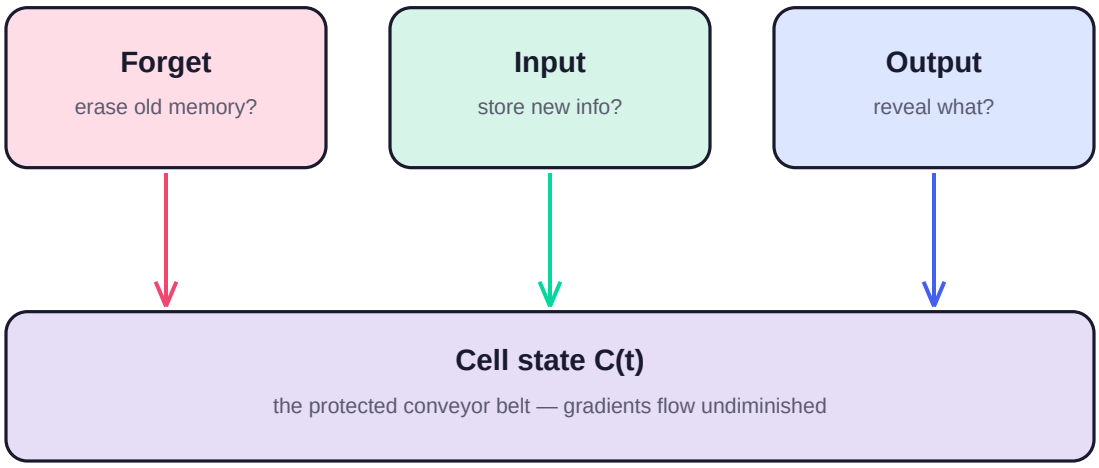

Each step multiplies the gradient by the recurrent weights. Values < 1 decay to zero exponentially.

Result: plain RNNs cannot learn long-range dependencies. Gradient clipping tames explosions; LSTMs fix vanishing.

STEP 4/5

# LSTM Gates

Three gates guard a protected cell state.

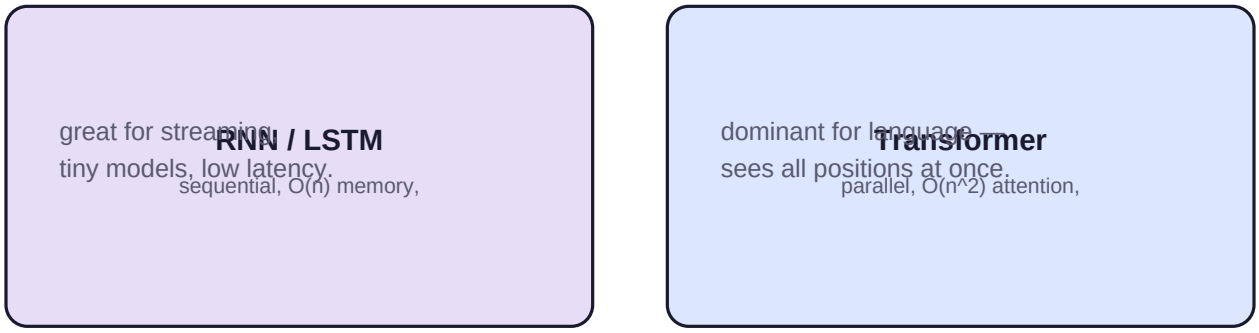



Gates are sigmoids (0-1) learned from data. The cell state is an additive highway — the fix for vanishing gradients.

STEP 5/5

# RNNs vs Transformers

When each architecture wins.




- RNNs process step-by-step: slow to train, but constant memory and natural for streaming.
- Transformers attend to everything in parallel: fast to train on GPUs, but memory grows quadratically.
- For long text, transformers won. For tiny on-device streaming models, RNNs/LSTMs still ship.

STEP ✓I✓

# Key Takeaways

## RNNs & LSTMs — Remember

- RNNs reuse weights across timesteps; the hidden state is the memory.
- Unrolling gives backprop-through-time (BPTT) — a very deep chain.
- Vanishing gradients kill long-range learning in plain RNNs.
- LSTM gates (forget/input/output) protect a cell-state highway for gradients.
- Transformers dominate language; RNNs still win for streaming and tiny models.

**Keep exploring**

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev